

WIP - Git Branching and Release Strategy

Executive Recommendation

Adopt a **trunk-based development model with controlled release branches and immutable artifact promotion.**

The central principle is:

Branches control code integration; GitLab environments control deployment promotion.

Do not maintain permanent `qa`, `uat`, and `prod` branches. Permanent environment branches tend to diverge, create repeated merge conflicts, and make it difficult to prove that QA, UAT, and production tested the same software.

Instead:

1. Merge completed features into `main`.
2. Test individual features using ephemeral EKS environments.
3. Create a temporary `release/*` branch containing only the features selected for a release.
4. Build the release image once and store it in ECR.
5. Promote the exact same ECR image from QA to UAT to production.
6. Use GitLab protected environments and deployment approvals as promotion gates.
7. Roll back by redeploying the previous ECR image.
8. Apply QA hotfixes to the release branch and immediately propagate the fix back to `main`.

1. Target Operating Model

Five Core Design Decisions

Decision	Recommendation	Rationale
Primary integration branch	Use one protected <code>main</code> branch	Maintains a single source of truth
Feature development	Use short-lived <code>feature/*</code> branches	Isolates development without creating permanent divergence

Independent feature testing	Deploy each merge request to an ephemeral EKS namespace	Allows features to be tested independently
Release composition	Create temporary <code>release/<version></code> branches	Enables controlled selection of features for QA
Environment promotion	Promote one immutable ECR image through QA, UAT, and production	Ensures every environment receives the same artifact

2. Branch Structure

```

1  main
2    └─ feature/TICKET-101-new-api
3    └─ feature/TICKET-102-scoring-change
4    └─ bugfix/TICKET-103-validation
5    └─ release/26.3.0
6        └─ hotfix/TICKET-110-qa-fix
7
8  Tags:
9  v26.3.0-rc.1
10 v26.3.0-rc.2
11 v26.3.0

```

`main`

`main` is the authoritative source for all completed engineering work.

Rules:

- Protected branch.
- No direct pushes.
- Changes enter through merge requests.
- CI pipeline must pass before merge.
- Required engineering and code-owner approvals.
- Must remain deployable.
- Incomplete or inactive features should be controlled through feature flags.

`feature/*`

A short-lived branch for one independently deliverable change.

Examples:

```
1 feature/CG-1421-international-labeling
2 feature/CG-1430-new-spam-feature
```

Rules:

- Created from the latest `main`.
- Contains one feature or logical change.
- Automatically deployed to a temporary EKS namespace.
- Deleted after merge.
- Not used as a release or environment branch.

`release/<version>`

A temporary stabilization branch representing one planned release.

Examples:

```
1 release/26.3.0
2 release/26.3.1
```

Rules:

- Created from a known baseline commit or production tag.
- Contains only the features approved for the release.
- Used to build release-candidate images.
- Accepts only release fixes after QA begins.
- Deleted after release completion.
- Production history is retained through Git tags.

`hotfix/*`

A short-lived branch used to correct a release candidate or production release.

Examples:

```
1 hotfix/CG-1490-null-pointer
2 hotfix/INC-3021-restore-api-timeout
```

Issue Found In	Hotfix Branch Starts From
Feature environment	Feature branch or <code>main</code>
QA or UAT	Active <code>release/*</code> branch

Production	Production tag or active maintenance branch
------------	---

3. End-to-End Release Flow

Stage 1: Develop and Independently Test Features

```
1 | main
2 |   └─ feature/A → review-a.example.com
3 |   └─ feature/B → review-b.example.com
4 |   └─ feature/C → review-c.example.com
```

Each merge request pipeline should:

1. Run linting and static validation.
2. Run unit tests.
3. Run security and dependency scans.
4. Build a temporary container image.
5. Push the image to ECR.
6. Create or update an EKS review namespace.
7. Deploy the feature using Helm or Kustomize.
8. Run integration and smoke tests.
9. Publish the review environment URL in GitLab.
10. Remove the review environment when the merge request closes or expires.

Example image:

```
1 | application:mr-184-<commit-sha>
```

Example namespace:

```
1 | review-mr-184
```

Feature Status Model

A feature can be:

- Code complete and merged into `main`.
- Operationally disabled through a feature flag.
- Excluded from a release by not selecting its commit.
- Included in a release but inactive through a feature flag.

Feature flags should be preferred when features share dependencies. Commit-level selection should be used for relatively independent changes.

Stage 2: Select Features for the QA Release

At release planning, the release manager selects approved features.

Feature	Status in <code>main</code>	Include in 26.3.0?
Feature A	Merged	Yes
Feature B	Merged	No
Feature C	Merged	Yes
Defect D	Merged	Yes

For a selective release, create the release branch from the current production baseline:

```
1 git fetch origin --tags
2
3 git switch --create release/26.3.0 v26.2.0
4
5 git cherry-pick <feature-a-merge-commit>
6 git cherry-pick <feature-c-merge-commit>
7 git cherry-pick <defect-d-merge-commit>
8
9 git push --set-upstream origin release/26.3.0
```

Release Construction Decision

Release Situation	Preferred Method
Small number of selected features	Start from production and cherry-pick selected changes
Most changes in <code>main</code> are approved	Branch from <code>main</code> ; disable incomplete features
Features have many shared dependencies	Branch from <code>main</code> and use feature flags
Emergency patch release	Start from the production tag

Release Manifest

Every release should include a machine-readable manifest:

```
1 release: 26.3.0
2 baseline: v26.2.0
3
4 features:
5   - id: CG-1421
```

```
6     merge_request: 184
7     commit: 73e0a37
8     enabled: true
9
10    - id: CG-1430
11      merge_request: 191
12      commit: 9ad732b
13      enabled: false
```

The manifest provides an auditable answer to:

- What is included?
 - What is excluded?
 - Which commit introduced each feature?
 - Which feature flags are enabled?
 - What was approved?
-

Stage 3: Build Once

After the release branch pipeline succeeds:

```
1 | release/26.3.0
2 |   |
3 |   ▼
4 | ECR image: application:26.3.0-rc.1
5 | Digest: sha256:8f93...
```

The pipeline must capture and promote the image digest rather than rebuilding the image separately for each environment.

Recommended identifiers:

```
1 | Human-readable tag:
2 | application:26.3.0-rc.1
3 |
4 | Immutable reference:
5 | <account>.dkr.ecr.<region>.amazonaws.com/application@sha256:8f93...
```

The promotion unit should be:

```
1 | Image digest + deployment configuration version
```

The promotion unit should not be:

```
1 | Branch name
```

Stage 4: Deploy to QA

The release candidate pipeline deploys the ECR image to QA:

```
1 | application@sha256:8f93... → QA EKS namespace or cluster
```

QA deployment should require:

- Successful release pipeline.
- Required automated tests.
- Release manager authorization.
- Serialized deployment.
- Post-deployment smoke tests.

Example GitLab job:

```
1 deploy_qa:
2   stage: deploy
3   resource_group: qa
4   environment:
5     name: qa
6     deployment_tier: testing
7   script:
8     - ./deploy.sh qa "$IMAGE_DIGEST"
9     - ./smoke-test.sh qa
```

Stage 5: Promote QA to UAT

QA approval should promote the already-tested artifact, not merge code into another branch.

```
1 application@sha256:8f93...
2   QA
3   |
4   QA approval
5   ▼
6   UAT
```

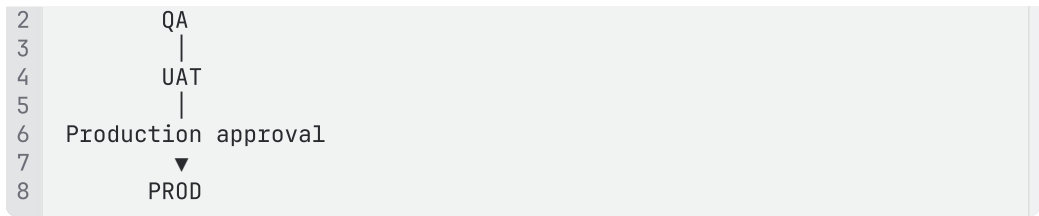
Recommended UAT gate:

Control	Owner
QA exit criteria completed	QA lead
Known defects accepted	Product owner
Release manifest validated	Release manager
Security exceptions approved	Security, when applicable
UAT deployment approved	Business or UAT owner

Stage 6: Promote UAT to Production

Production receives the same image digest that passed QA and UAT.

```
1 application@sha256:8f93...
```



Recommended production gate:

- UAT acceptance completed.
- Change record approved.
- Release notes completed.
- Rollback procedure verified.
- Monitoring dashboard prepared.
- Database migration compatibility confirmed.
- Release manager approval.
- Production owner approval.
- Security or change authority approval where required.

After successful production deployment:

```
1 | git tag -a v26.3.0 <release-commit> -m "Production release 26.3.0"
2 | git push origin v26.3.0
```

ECR tags may be updated for readability:

```
1 | application:26.3.0
2 | application:prod
3 | application@sha256:8f93...
```

The digest remains the authoritative identifier.

4. Approval and Protection Model

Recommended Control Matrix

Environment	Automatic Deployment	Required Approval	Authorized Deployers
Feature/review	Yes	None	CI service identity
Development	Yes from <code>main</code>	None	CI service identity

QA	Optional or manual	Release manager or QA lead	Release engineering
UAT	No	QA lead and product owner	Release engineering
Production	No	Product owner and operations/change authority	Restricted production deployers

GitLab Configuration

Protect:

- `main`
- `release/*`
- `hotfix/*`
- Production tags such as `v*`

Configure:

- No direct push to `main`.
- Merge request required.
- Pipeline must succeed.
- Code-owner approvals for sensitive files.
- Author cannot self-approve.
- Approvals reset when new commits are added.
- QA, UAT, and production environments are protected.
- Environment-specific deployment approvers.
- Environment-scoped CI/CD variables.
- Separate AWS IAM roles for each environment.

5. Rollback Strategy

Rollback should be available at three levels.

Level 1: Feature Disablement

Use when the release is stable but one capability is defective.

Advantages:

- Fastest recovery.
- No container rollback.
- Limits impact to one feature.
- Preserves unrelated fixes in the current release.

Use feature flags for:

- New algorithms.
- New APIs.
- New integrations.
- Customer-visible behavior changes.
- High-risk processing paths.

Level 2: Application Rollback

Redeploy the previous successful ECR image digest:

```
1 Current:
2 application@sha256:new
3
4 Rollback:
5 application@sha256:previous
```

Example:

```
1 rollback_production:
2   stage: rollback
3   when: manual
4   resource_group: production
5   environment:
6     name: production
7   script:
8     - ./deploy.sh production "$PREVIOUS_PRODUCTION_DIGEST"
9     - ./smoke-test.sh production
```

Level 3: Code Revert

Use when the defective change must be removed from future releases:

```
1 git switch main
2 git pull --ff-only
3 git switch --create revert/C6-1421
4 git revert <merge-commit>
5 git push --set-upstream origin revert/C6-1421
```

Submit the revert through a normal merge request.

Database Requirement

Every database change should support at least one of the following:

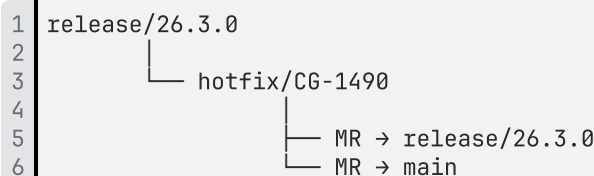
- Backward-compatible application rollback.

- Expand-and-contract migration.
- Tested down migration.
- Forward-fix recovery plan.

A container rollback is unsafe when an irreversible schema change makes the previous application version incompatible.

6. QA Hotfix Process

Required Flow



Procedure

1. Create the hotfix from the active release branch:

```

1 | git switch release/26.3.0
2 | git pull --ff-only
3 |
4 | git switch --create hotfix/CG-1490
  
```

2. Implement and test the fix.
3. Merge the hotfix into `release/26.3.0`.
4. Build a new release candidate:

```

1 | application:26.3.0-rc.2
  
```

5. Deploy `rc.2` to QA.
6. Propagate the same fix to `main`:

```

1 | git switch main
2 | git pull --ff-only
3 | git switch --create forward-fix/CG-1490
4 | git cherry-pick <hotfix-commit>
5 | git push --set-upstream origin forward-fix/CG-1490
  
```

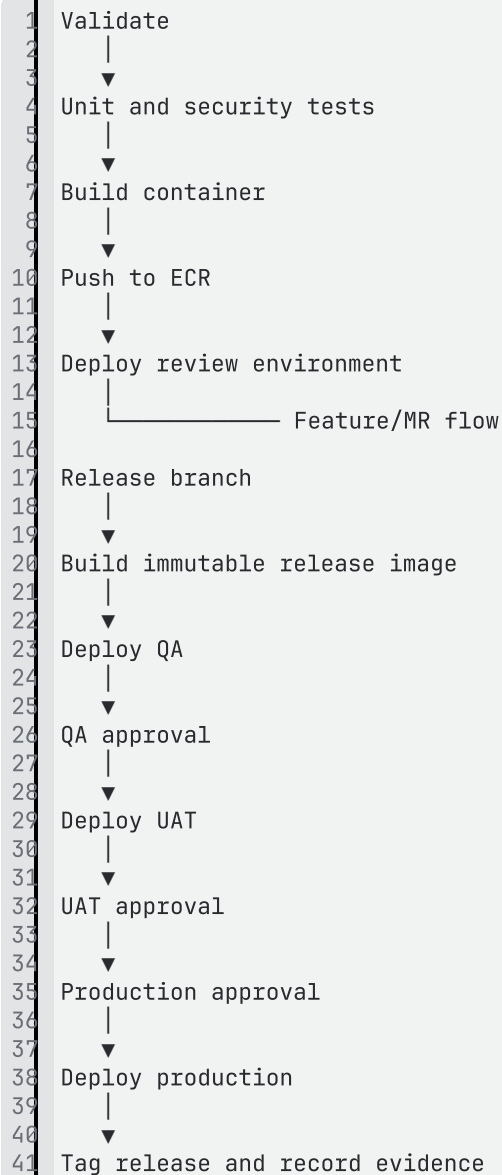
7. Open a merge request from `forward-fix/CG-1490` to `main`.

Mandatory Control

A hotfix is not complete until it exists in both the active release line and `main`.

This prevents the same defect from returning in a future release.

7. CI/CD Pipeline Structure



Suggested GitLab Stages

```
1  stages:
2    - validate
3    - test
4    - security
5    - build
6    - publish
7    - deploy_review
8    - deploy_qa
9    - verify_qa
10   - deploy_uat
11   - verify_uat
12   - deploy_prod
13   - verify_prod
14   - rollback
```

Simplified Promotion Pattern

```
1  variables:
```

```

2  IMAGE_REPOSITORY:
  "$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/my-app"
3
4  deploy_qa:
5    stage: deploy_qa
6    resource_group: qa
7    environment:
8      name: qa
9    script:
10     - ./deploy.sh qa "$IMAGE_DIGEST"
11     - ./smoke-test.sh qa
12    rules:
13     - if: '$CI_COMMIT_BRANCH =~ /^release\/.+/ '
14
15  deploy_uat:
16    stage: deploy_uat
17    resource_group: uat
18    environment:
19      name: uat
20    script:
21     - ./deploy.sh uat "$IMAGE_DIGEST"
22     - ./smoke-test.sh uat
23    needs:
24     - deploy_qa
25    rules:
26     - if: '$CI_COMMIT_BRANCH =~ /^release\/.+/ '
27
28  deploy_prod:
29    stage: deploy_prod
30    resource_group: production
31    environment:
32      name: production
33    script:
34     - ./deploy.sh production "$IMAGE_DIGEST"
35     - ./smoke-test.sh production
36    needs:
37     - deploy_uat
38    rules:
39     - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/ '

```

Formal deployment approvals should be implemented through GitLab protected environments rather than relying only on `when: manual` .

8. AWS and EKS Deployment Boundaries

Recommended Isolation

Component	Development	QA	UAT	Production
AWS IAM role	Separate	Separate	Separate	Separate
EKS namespace	Dedicated	Dedicated	Dedicated	Dedicated
ECR repository	Shared or non-prod	Shared or non-prod	Shared or promoted	Production controlled

Secrets	Environment scoped	Environment scoped	Environment scoped	Environment scoped
GitLab runner	Non-prod runner	Non-prod runner	Controlled runner	Restricted production runner
Approval authority	Engineering	QA/release	Business/QA	Operations/c hange authority

Each GitLab deployment job should assume only the AWS role required for its target environment.

Use GitLab OIDC federation with AWS STS rather than persistent AWS access keys.

EKS access should follow least privilege across:

- AWS IAM permissions.
- Kubernetes RBAC.
- Namespace scope.
- Deployment-specific permissions.

9. Operating Guardrails

Team Should

- Keep feature branches short-lived.
- Merge completed code into `main`.
- Use feature flags for incomplete or high-risk capabilities.
- Build release artifacts once.
- Promote image digests rather than rebuilding.
- Use temporary release branches.
- Propagate every hotfix to `main`.
- Tag every production release.
- Maintain release manifests.
- Make database changes backward compatible.

Team Should Not

- Maintain permanent `qa`, `uat`, and `prod` code branches.

- Merge QA into UAT and UAT into production.
 - Rebuild containers during promotion.
 - Deploy mutable tags such as only `latest`.
 - Patch production manually.
 - Commit environment secrets into Git.
 - Fix only the release branch without updating `main`.
 - Keep large feature branches open across multiple release cycles.
-

10. Implementation Roadmap

Phase 1: Establish Governance

1. Protect `main`, `release/*`, and production tags.
2. Configure merge request approval rules.
3. Configure `CODEOWNERS`.
4. Prevent direct pushes.
5. Require successful pipelines before merge.
6. Define release manager, QA, UAT, and production approver groups.

Phase 2: Standardize CI

1. Create a reusable GitLab CI template.
2. Standardize container build and ECR publishing.
3. Tag every image with the commit SHA.
4. Capture the ECR image digest as a pipeline artifact.
5. Add security, unit, integration, and smoke-test stages.
6. Configure serialized deployment resource groups.

Phase 3: Introduce Independent Feature Environments

1. Define a Helm chart or Kustomize base.
2. Deploy each merge request into an isolated EKS namespace.
3. Generate unique ingress hostnames.
4. Add automated cleanup.
5. Add test-data isolation.
6. Publish review URLs in merge requests.

Phase 4: Establish Release Promotion

1. Introduce `release/<version>` branches.

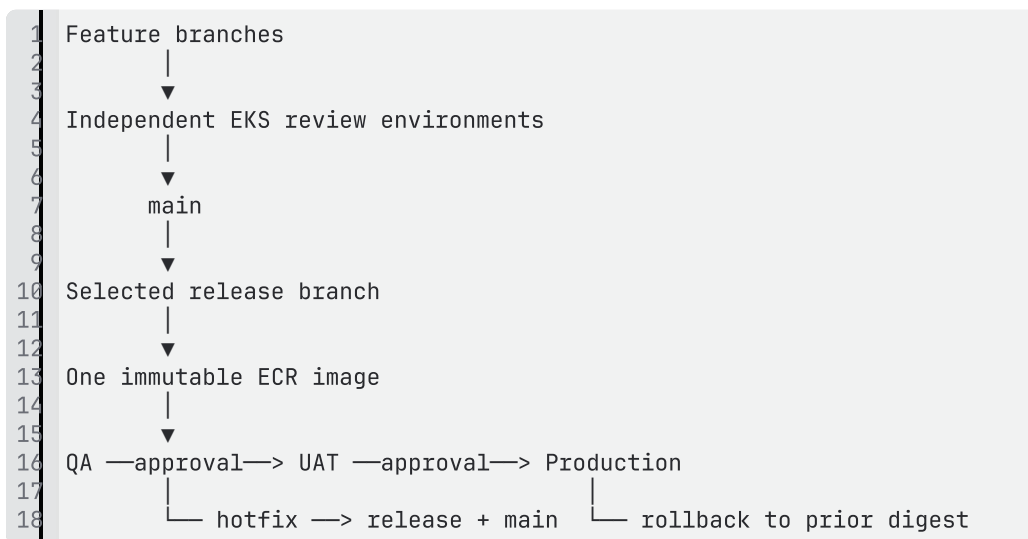
2. Add release manifest generation.
3. Build one release-candidate image.
4. Configure QA, UAT, and production GitLab environments.
5. Protect each environment.
6. Configure deployment approval rules.
7. Promote one image digest through all environments.

Phase 5: Operationalize Recovery

1. Add a feature-flag standard.
2. Add rollback jobs for every environment.
3. Record the last successful image digest.
4. Test rollback regularly.
5. Establish backward-compatible database migration standards.
6. Automate the requirement to propagate hotfixes to `main`.

Team Summary

Develop independently, integrate continuously, compose releases deliberately, and promote immutable artifacts.



This model provides flexibility while keeping the release process controlled, auditable, and recoverable.